

Gymnázium, Praha 6, Arabská 14

Obor Programování

Ročníková práce

Bludiště (Maze)



Prohlašuji, že jsem jediným autorem tohoto projektu, všechny citace jsou řádně označené a všechna použitá literatura a další zdroje jsou v práci uvedené. Tímto dle zákona 121/2000 Sb. (tzv. Autorský zákon) ve znění pozdějších předpisů uděluji bezúplatně škole Gymnázium, Praha 6, Arabská 14 oprávnění k výkonu práva na rozmnožování díla (§ 13) a práva na sdělování díla veřejnosti (§ 18) na dobu časově neomezenou a bez omezení územního rozsahu.

V Praze dne 30. dubna 2026 (vlastnoruční podpis)

Anotace Tento projekt se zabývá vývojem 2D počítačové hry typu bludiště, vytvořené v programovacím jazyce Java s využitím grafické knihovny Swing. Cílem hry je navigovat hráče skrze tři úrovně s odlišnými funkcemi – od jednoduchého hledání cesty přes vyhýbání se pohyblivým nepřátelům až po zvládnutí úrovně s časovanými projektily. Práce popisuje použití mřížkového systému (grid-based movement), detekci kolizí a správu dynamických objektů v herní smyčce.

Abstract This project focuses on the development of a 2D maze game created in Java using the Swing library. The game features three levels with increasing difficulty and different mechanics, such as moving enemies and timed projectiles. The documentation describes the usage of a grid-based movement system, collision detection, and dynamic object management within the game loop.

Obsah

1. Výběr projektu a zadání.....	4
1.1. Proč právě bludiště?.....	4
1.2. Cíle zadání.....	4
2. Postup práce.....	5
2.1. Fáze přípravy.....	5
2.2. Vývojový proces.....	5
3. Použité technologie a odborné pojmy.....	6
3.1. Vývojové prostředí a nástroje.....	6
3.2. Vysvětlení odborných pojmů.....	6
3.3. Vnější rozhraní.....	6
4. Popis problematiky.....	7
4.1. Klíčové výzvy.....	7
5. Klíčové algoritmy a řešení problémů.....	8
5.1. Mechanismus pohybu a detekce kolizí.....	8
5.2. Herní smyčka a plynulost animací.....	8
5.3. Regulace rychlosti pomocí operátoru modulo.....	9
5.4. Logika projektilů s omezeným doletem.....	9
5.5. Matematické výpočty a transformace souřadnic.....	10
5.6. Výpočet vzdálenosti a omezení doletu.....	11
5.7. Změna směru pohybu.....	11
6. Uživatelská příručka a instalace.....	12
6.1. Systémové požadavky.....	12
6.2. Instalace a spuštění.....	12
6.3. Ovládání hry.....	12
7. Zhodnocení a závěr.....	13
7.1. Dosažené výsledky.....	13
7.2. Problémy při řešení.....	14
7.3. Možnosti dalšího rozvoje.....	14
8. Seznam použitých zdrojů.....	15
8.1. Seznam použité literatury.....	15
8.2. Použité prompty při práci s AI.....	16

1. Výběr projektu a zadání

1.1. Proč právě bludiště?

Při výběru ročníkového projektu jsem hledal téma, které by propojilo logické myšlení s vizuálním výsledkem. Hry typu „Maze“ (bludiště) jsou ideálním příkladem, jak si procvičit práci s poli a algoritmy pro pohyb v uzavřeném prostoru. Chtěl jsem vytvořit něco, co nebude jen statickým obrázkem, ale bude reagovat na uživatele a postupně mu představovat nové výzvy.

1.2. Cíle zadání

Moje původní vize, kterou jsem si nechal schválit, obsahovala tyto body:

- Vytvořit funkční grafické okno v jazyce Java.
- Implementovat pohyb hráče ovládaný klávesnicí.
- Vytvořit systém úrovní (levelů), které se načítají po dosažení cíle.
- Přidat do hry interaktivní prvky, jako jsou pohybliví nepřátelé a nebezpečné zóny.

2. Postup práce

2.1. Fáze přípravy

Práci jsem začal studiem grafické knihovny **Swing**. Nejdříve jsem zkoušel tvořit různá okna a zkoušel na něj vykreslit první čtverec, který reprezentoval hráče. Hlavním rozhodnutím bylo použití mřížkového systému – uvědomil jsem si, že je mnohem jednodušší počítat kolize v tabulce (poli), než hlídat každý pixel na obrazovce zvlášť.

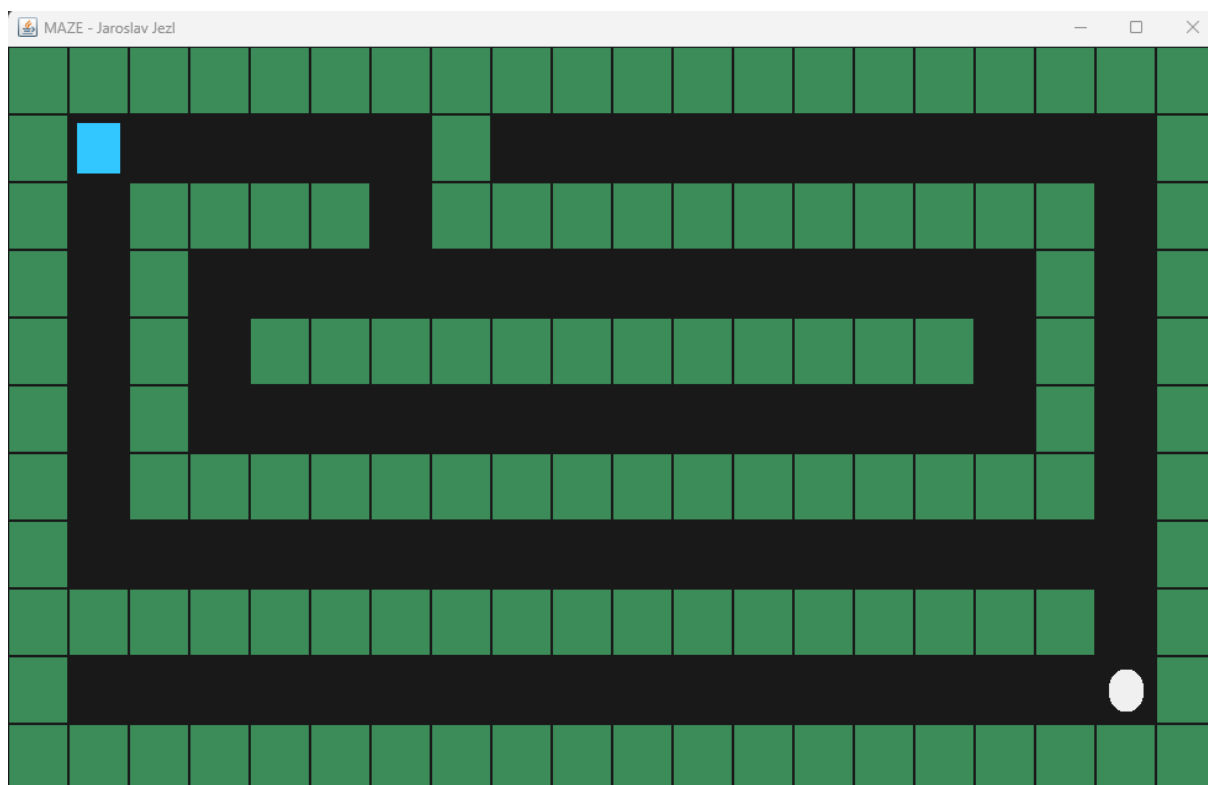
2.2. Vývojový proces

Základní pohyb: Rozchození ovládání šipkami a zablokování průchodu skrz „zdi“.

Tvorba map: Návrh tří úrovní v podobě číselných polí (0 a 1...).

Animace a nepřátelé: Přidání třídy Ryba (pro level 2), kde jsem se naučil používat herní časovač (Timer).

Komplexní mechaniky: Vytvoření systému střelby v levelu 3. Zde bylo nejtěžší bylo vytvořit ty střely a jak je mazat z paměti, když doletí do zdi, aby program nezamrzl.



Obrázek 1 z levelu 1

3. Použité technologie a odborné pojmy

3.1. Vývojové prostředí a nástroje

Pro realizaci projektu jsem si zvolil technologie, které umožňují lehčí vývoj desktopových aplikací s grafickým rozhraním:

Java (verze 17/21): Hlavní programovací jazyk. Vybral jsem si Java, kvůli tomu že ji používáme na hodinách a protože má silnou kontrolu a vestavěný knihovny pro tvorbu her.

IntelliJ IDEA: Toto prostředí jsem si vybral především proto, že ho používáme i ve škole, takže jsem s ním už obeznámený a dobře se mi v něm pracuje. Díky tomu jsem nemusel ztrácet čas učením nového programu a mohl jsem se soustředit přímo na samotnou práci.

Dalším důvodem výběru je to, že IntelliJ IDEA nabízí přehledné uživatelské prostředí a spoustu užitečných funkcí, jako je například automatické doplňování kódu, zvýrazňování chyb nebo jednoduchá orientace v projektu.

Java Swing & AWT: Knihovny použité pro vykreslování herního světa. Swing zajišťuje vytvoření okna aplikace, zatímco AWT poskytuje nástroje pro práci s barvami a grafikou.

3.2. Vysvětlení odborných pojmů

Aby byl text srozumitelný i pro člověka který nemá velkou vědomost programování, jsou zde definovány klíčové termíny použité v další části práce:

OOP (Objektově orientované programování): Styl programování, ve kterém jsou jednotlivé části hry (hráč, mapa, nepřátelé) chápány jako samostatné objekty.

Double-buffering: Technika, která zabraňuje blikání obrazovky. Obraz se nejdříve připraví v paměti a až poté se naráz zobrazí uživateli.

Herní smyčka (Game Loop): Mechanismus řízený časovačem, který neustále aktualizuje stav hry a překresluje scénu (v našem případě 20x za sekundu).

3.3. Vnější rozhraní

Komunikace mezi uživatelem a programem probíhá přes rozhraní klávesnice. Program využívá standardní API Javy (KeyListener), které zachytává vstupy z klávesnice a převádí je na akce v herním světě. To umožňuje plynulé ovládání postavy šipkami v reálném čase.

4. Popis problematiky

4.1. Klíčové výzvy

Během tvorby jsem narazil na několik problémů. Největší výzvou bylo blikání obrazovky při překreslování. To jsem vyřešil použitím metody **paintComponent** a správným voláním **repaint()**, což v Javě zajišťuje tzv. **double-buffering** (hladké překreslování). Dalším problémem byla synchronizace rychlosti hry na různých počítačích, což vyřešil fixní **interval Timeru**. A dále jsem měl problémy s tím, že se postava hráče byla schopna projít zdmi, a to jsem opravil tím, že jsem do obsluhy klávesnice implementoval **algoritmus predikce pohybu**. Místo okamžité aktualizace souřadnic hráče program nejdříve vypočítá potenciální novou pozici do pomocných proměnných. Následně proběhne kontrola v dvourozměrném poli mapy na těchto souřadnicích. Pokud je v poli detekována hodnota 1 (reprezentující zeď), je pohybový příkaz ignorován a souřadnice hráče zůstanou nezměněny. Hráč se tedy pohne pouze v případě, že cílové políčko v poli obsahuje hodnotu 0 (cesta) nebo 2 (cíl).

5. Klíčové algoritmy a řešení problémů

5.1. Mechanismus pohybu a detekce kolizí

Základním kamenem hry je algoritmus, který zajišťuje, aby se hráč mohl pohybovat pouze po cestě a neprocházel zdmi. Místo abychom kontrolovali každý pixel na obrazovce, využíváme indexy v dvourozměrném poli.

Algoritmus pracuje ve třech krocích:

1. **Predikce:** Při stisku klávesy se do dočasných proměnných vypočítá nová pozice, na kterou chce hráč vstoupit (např. proměnné *r* a *s*).
2. **Validate:** Program se podívá do pole rozložení na vypočítané souřadnice. Pokud je hodnota na těchto souřadnicích rovna 1 (stěna), pohyb neproběhne.
3. **Realizace:** Pouze pokud je cesta volná (hodnota 0 nebo 2), přepíše se skutečné souřadnice hráče těmi dočasnými.

```
// Ovládání klávesnicí
addKeyListener(new KeyAdapter() {
    @Override
    public void keyPressed(KeyEvent e) {
        int r = hracR;
        int s = hracS;

        // Výpočet potenciální nové pozice
        if (e.getKeyCode() == KeyEvent.VK_UP) r--;
        if (e.getKeyCode() == KeyEvent.VK_DOWN) r++;
        if (e.getKeyCode() == KeyEvent.VK_LEFT) s--;
        if (e.getKeyCode() == KeyEvent.VK_RIGHT) s++;

        // Potvrzení pohybu proti mřížce pole
        if (r >= 0 && r < 11 && s >= 0 && s < 20 && plan.rozlozeni[r][s]
!= 1) {
            hracR = r;
            hracS = s;

            // Kontrola cíle
            if (plan.rozlozeni[r][s] == 2) {
                dalsiLvl(cislo_lvl + 1);
            }
        }
    }
});
```

5.2. Herní smyčka a plynulost animací

Pro zajištění plynulosti hry byla použita třída **javax.swing.Timer**. Ten v pravidelném intervalu 50 milisekund (což odpovídá 20 snímkům za sekundu) vykonává dvě klíčové operace:

Update: Přepočítá pozice pohyblivých nepřátel a střel.

Render: Vyvolá metodu **repaint()**, která smaže starý obraz a vykreslí nový stav všech objektů.

```
new Timer(50, e -> {
    casovac++; // počítadlo pro zpomalení akcí

    repaint(); // příkaz k překreslení grafiky na obrazovce
}).start();
```

5.3. Regulace rychlosti pomocí operátoru modulo

V projektu byl vyřešen problém s různou rychlostí objektů v rámci jedné smyčky. Pomocí operátoru modulo (%) a čítače **casovac** se určité akce (např. pohyb ryby) provádějí pouze v každém n-tém tik **Timeru**. To umožňuje, aby se hra překreslovala rychle a plynule, zatímco nepřátelé se pohybují rychlostí, kterou hráč dokáže vnímat a reagovat na ni, což je těch 20 snímků za vteřinu.

```
// Ryba se pohne pouze v každém čtvrtém tik časovače

if (cislo_lvl == 2 && casovac % 4 == 0) {
    for (Ryba r : nepratele) {
        r.pohyb(plan.rozlozeni);
        // kontrola kolize ryby s hráčem

        if (r.r == hracR && r.s == hracS) { hracR = 1; hracS = 1; }
    }
}
```

5.4. Logika projektilů s omezeným doletem

V třetí úrovni byl použit systém střelby. Klíčovým algoritmem je zde kontrolování doletu střely. Každý objekt třídy **Koule** si nese informaci o počtu uražených políček (**let**). Jakmile hodnota dosáhne limitu(5 bloků) nebo narazí na zeď, je objekt odstraněn z dynamického seznamu **ArrayList**, čímž se uvolní prostředky v operační paměti a ulehčí tak procesoru.

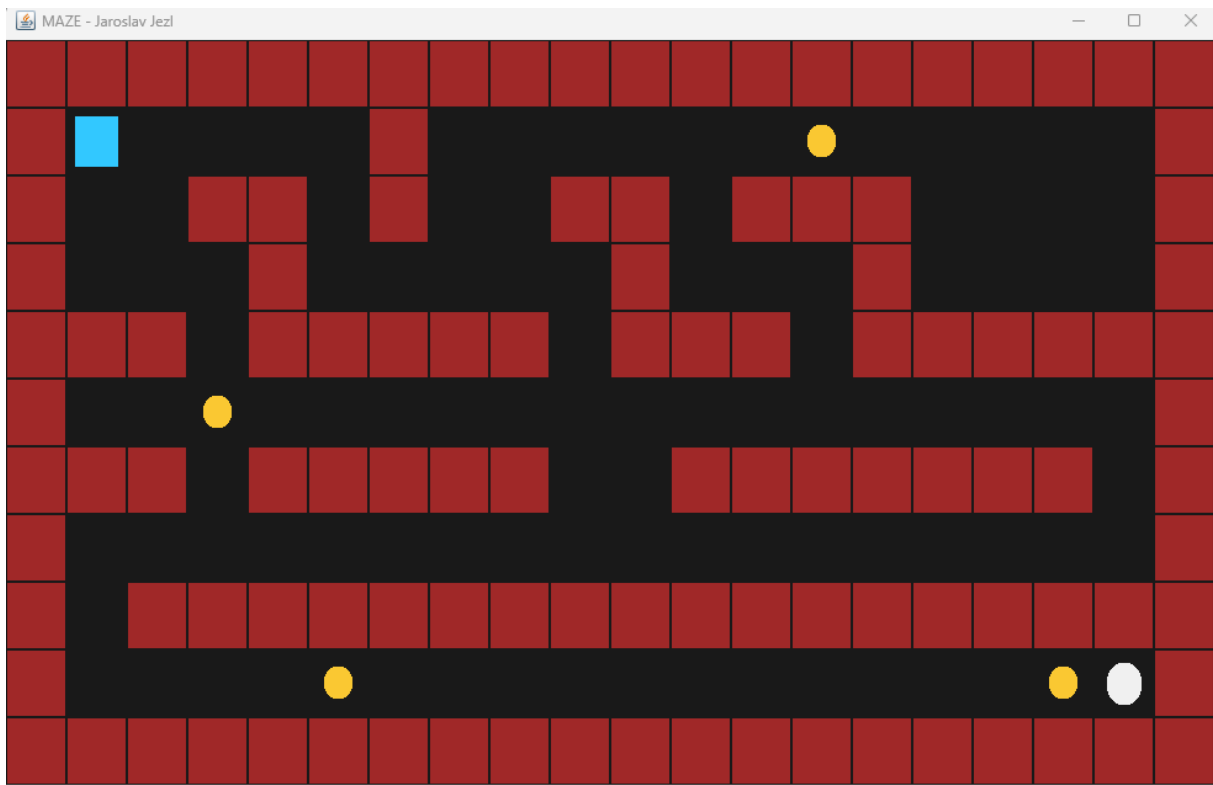
```
// Pohyb a správa střel v ArrayListu
for (int i = naboje.size() - 1; i >= 0; i--) {
    Koule k = naboje.get(i);
    k.s += k.smer; // posun střely o sloupec
    k.let++;       // zvýšení proměnné o 1 počítadla doletu

    // Kontrola kolize střely s hráčem
    if (k.r == hracR && k.s == hracS) restartujPozici();
}
```

```

4.      // Odstranění střely při nárazu do zdi nebo po doletu 5 políček
5.      if (k.let >= 5 || k.s < 0 || k.s >= 20 || plan.rozlozeni[k.r][k.s]
    == 1) {
6.          naboje.remove(i);
7.      }
    }      naboje.remove(i);
    }
}

```



Obrázek 2 z levelu 3

5.5. Matematické výpočty a transformace souřadnic

Přepočet souřadnic (Mřížka na Pixely) Hra běží ve dvou světech. Prvním je **logická mřížka** (tabulka, kde jsou zapsány zdi a cesta), druhým je **obrazovka monitoru** (pixels). Aby program věděl, kam přesně nakreslit čtvereček, používá jednoduché násobení:

Pozice X = (číslo sloupce v tabulce) × (šířka jednoho políčka)

Pozice Y = (číslo řádku v tabulce) × (výška jednoho políčka) Tím je zajištěno, že ať je postava na jakémkoliv políčku, vždy se vykreslí přesně tam, kam patří.

```
// Vykreslení hráče
g2.setColor(new Color(0, 153, 255)); // Definice barvy hráče
int offset = 8; // Mezera od stěn políčka

g2.fillRoundRect(
    hracS * plan.sirka_ctverce + offset,
    hracR * plan.vyska_ctverce + offset,
    plan.sirka_ctverce - (offset * 2),
    plan.vyska_ctverce - (offset * 2),
    10, 10
);
```

5.6. Výpočet vzdálenosti a omezení doletu

Místo složitých goniometrických výpočtů používá střelba ve třetí úrovni jednoduché celočíselné počítadlo let.

Každý náboj si ukládá počet políček, o která se posunul. Jakmile tato hodnota dosáhne limitu 5, je projektil považován za vyhaslý a je odstraněn ze seznamu aktivních objektů. Tím se řeší, že projektily se nemůžou dotknout hráče přes celou mapu.

5.7. Změna směru pohybu

Nepřátelé (ryby) se pohybují pomocí proměnné **směr**, která má hodnotu buď **1** (doprava), nebo **-1** (doleva).

- Když ryba narazí do zdi, program jednoduše vynásobí směr číslem **-1**.
- Tím se z jedničky stane mínus jednička (a naopak), což rybu okamžitě otočí na druhou stranu.

```
// Pokud ryba narazí na překážku, vynásobí směr číslem -1 (otočí se)

if (s + strana >= 20 || s + strana < 0 || m[r][s + strana] == 1) {
    strana *= -1;
} else {
    s += strana; // jinak pokračuje v aktuálním směru
}
```

6. Uživatelská příručka a instalace

6.1. Systémové požadavky

Program je napsán v jazyce Java, což zajišťuje jeho multiplatformnost. Pro spuštění je vyžadováno:

- **Operační systém:** Windows, macOS nebo Linux.
- **Běhové prostředí:** Java Runtime Environment (JRE) verze 17 nebo novější.
- **Hardware:** Standardní PC s klávesnicí (vyžadována pro ovládání).

6.2. Instalace a spuštění

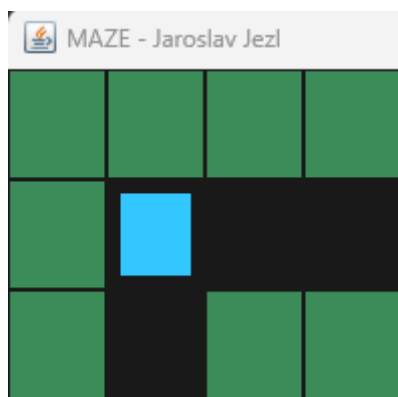
Hra je distribuována jako spustitelný soubor typu .jar, případně jako projekt se zdrojovými kódy.

1. **Spuštění z JAR:** Dvojklikem na soubor Maze.jar (pokud je v systému správně připojena Java).
2. **Spuštění z IDE:** Otevřením projektu v prostředí IntelliJ IDEA a spuštěním metody main ve třídě Hra.

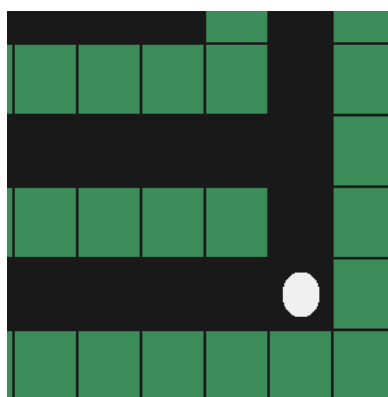
6.3. Ovládání hry

Pohyb hry je navržen co jednodušeji:

- **Pohyb postavy:** Směrové šipky (Nahoru, Dolů, Doleva, Doprava).
- **Cíl hry:** Projít bludištěm a dotknout se bílého objektu (cíl), který hráče automaticky přesune do další úrovně.
- **Ukončení:** Křížkem v rohu okna nebo dosažením konce poslední úrovně.



Obrázek 3 postava hráče

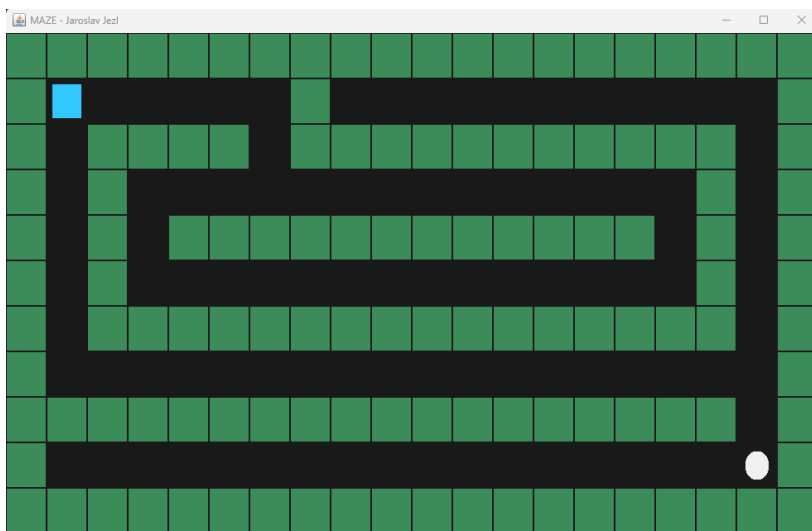


Obrázek 4 cíl

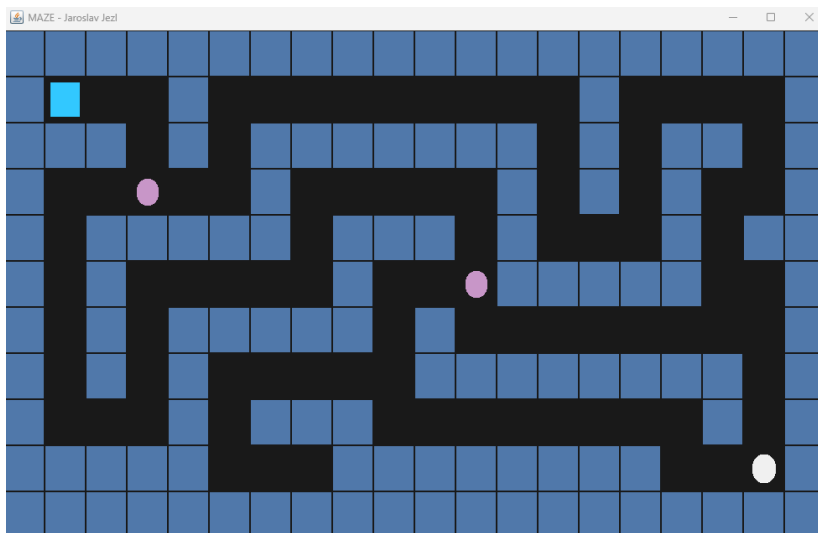
7. Zhodnocení a závěr

7.1. Dosažené výsledky

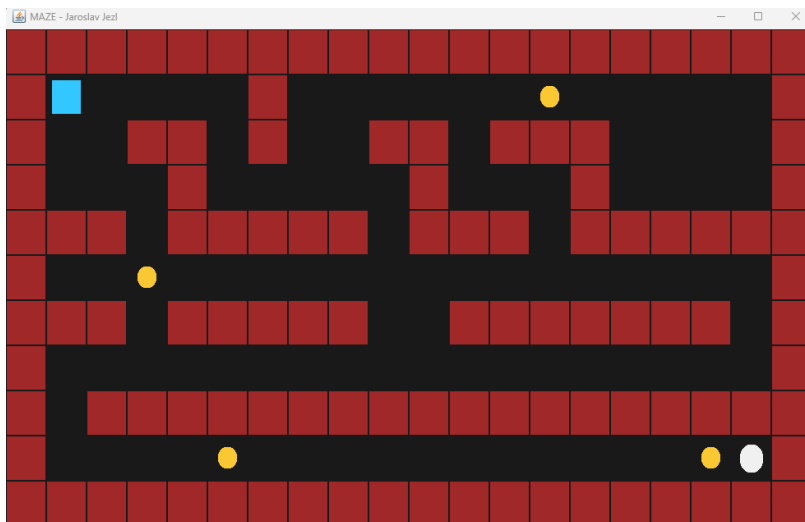
Cílem práce bylo vytvořit funkční 2D hru s několika úrovněmi a různými mechanikami. Tento cíl byl splněn. Podařilo se použít mřížkový systém pohybu, který je spolehlivý. Hra plynule přepíná mezi třemi levely, přičemž každý přináší nové prostředí, překážky a pasti.



Obrázek 5 level 1



Obrázek 6 level 2



Obrázek 7 level 3

7.2. Problémy při řešení

Během vývoje bylo největší výzvou správné nastavení herní smyčky tak, aby se objekty nehýbaly příliš rychle. Dalším problémem byla práce s dynamickým seznamem **ArrayList** pro střely v 3. úrovni, kde bylo nutné zajistit, aby se objekty po nárazu správně odstraňovaly a nezpůsobovaly chyby v paměti (**ConcurrentModificationException**).

7.3. Možnosti dalšího rozvoje

Přestože je aktuální verze hry plně funkční a splňuje všechny body zadání, existuje několik vylepšení, kterými by se dal projekt v budoucnu rozšířit:

- **Implementace herních předmětů:** Rozšíření herní logiky o systém sbírání předmětů (např. klíče k odemčení dveří, mince pro zvýšení skóre nebo bonusy zvyšující rychlost hráče anebo tajné cesty). To by vyžadovalo přidání dalších kódů do mřížkového pole a rozšíření kolizního algoritmu.
- **Zvukový doprovod:** Přidání audio stop pro různé herní události, jako je pohyb, náraz do nepřítele nebo úspěšné dokončení úrovně, pomocí tříd Clip a AudioSystem.
- **Implementace aktivního pronásledování:** Výrazným vylepšením by bylo přidání inteligentního nepřítele, který by v reálném čase reagoval na pozici hráče. Tento systém by využíval algoritmus pro hledání nejkratší, který by v každém kroku vyhodnotil okolní volná políčka mřížky a navedl nepřítele směrem k aktuálním souřadnicím hráče. To by do hry přidalo prvek napětí a zvýšilo celkovou obtížnost.

8. Seznam použitých zdrojů

8.1. Seznam použité literatury

CodeGym.cc. Tutoriál Java Swing pro začátečníky. [online]. [cit. 25. 4. 2026]. Dostupné z: <https://codegym.cc/groups/posts/java-jtable>

GeeksforGeeks.org. Práce s rozhraním KeyListener v Javě. [online]. [cit. 25. 4. 2026]. Dostupné z: <https://www.geeksforgeeks.org/java/java-keylistener-in-awt/>

GitHub.com. Dokumentace platformy GitHub – začínáme s repozitáři. [online]. [cit. 25. 4. 2026]. Dostupné z: <https://docs.github.com/en>

GameDev.net. Základy detekce kolizí ve 2D hrách. [online]. [cit. 25. 4. 2026]. Dostupné z: <https://gamedev.net/tutorials/programming/general-and-gameplay-programming/game-programming-beginners-guide-r906/>

JavaTpoint.com. Kompletní průvodce knihovnou Java Swing. [online]. [cit. 25. 4. 2026]. Dostupné z: <https://www.geeksforgeeks.org/java/introduction-to-java-swing/>

KhanAcademy.org. Matematika: Lineární transformace a matice. [online]. [cit. 25. 4. 2026]. Dostupné z: <https://www.khanacademy.org/math/linear-algebra>

Oracle.com. Dokumentace ke třídě Timer (Java SE 8). [online]. [cit. 25. 4. 2026]. Dostupné z: <https://docs.oracle.com/javase/8/docs/api/javax/swing/Timer.html>

Oracle.com. Oficiální tutoriál: Tvorba uživatelského rozhraní v Swing. [online]. [cit. 25. 4. 2026]. Dostupné z: <https://docs.oracle.com/javase/tutorial/uiswing/>

ProgramCreek.com. Příklady použití metody repaint v grafických komponentách. [online]. [cit. 25. 4. 2026]. Dostupné z: <https://stackoverflow.com/questions/58187624/how-to-reload-a-java-swing-gui>

StackOverflow.com. Technika Double buffering v knihovně Swing. [online]. [cit. 25. 4. 2026]. Dostupné z: <https://stackoverflow.com/questions/4430356/java-how-to-do-double-buffering-in-swing>

Study.com. Práce s dvourozměrnými poli v jazyce Java. [online]. [cit. 25. 4. 2026]. Dostupné z: <https://study.com/academy/lesson/how-to-sort-an-array-in-java.html>

W3Schools.com. Práce s dynamickým seznamem ArrayList. [online]. [cit. 25. 4. 2026]. Dostupné z: https://www.w3schools.com/java/java_arraylist.asp

Wikipedia.org. Informace o vývojovém prostředí IntelliJ IDEA. [online]. [cit. 25. 4. 2026]. Dostupné z: https://cs.wikipedia.org/wiki/IntelliJ_IDEA

8.2. Použité prompty při práci s AI

Použité prompty – Google Gemini (25. 4. 2026)

Prompt: Dělán v Javě hru s bludištěm. Jaké konkrétní metody a techniky by se mi hodily pro tyto části:

1. **Generování mapy:** Jak efektivně vykreslit bludiště z 2D pole pomocí cyklů?
2. **Kolize:** Jaká metoda je nejlepší pro kontrolu, zda hráč nenarazil do zdi?
3. **Pohyb:** Jak plynule posouvat postavu a jak řešit různé rychlosti objektů?
4. **Vizuál:** Jaké metody využít pro udělání designu a grafiky a výměna barev u jiných levelů?

Prompt: Vysvětli mi jako začátečníkovi, jak funguje metoda `@Override protected void paintComponent(Graphics g)`. Proč tam musí být to `@Override`, co přesně dělá ten objekt `Graphics g` a jak to použít pro udělání hry bludiště v Javě?

Prompt: Jak přesně funguje `new Timer(50, e -> { ... }).start();` v knihovně `Swing`? A jak to implementovat do hry bludiste, proč se tam používá lambda výraz `e ->` a jak tento časovač zajišťuje, že se hra hýbe plynule i bez zásahu uživatele.

Prompt: Vysvětli mi matematickou logiku řádku `if (casovac % 4 == 0)`. Jak mi operátor modulo pomůže v tom, aby se jedna postava (třeba ryba) pohybovala pomaleji než zbytek hry, i když celá hra běží na stejné snímkové frekvenci?

Prompt: Mám ve hře `ArrayList< Koule >` naboje. Vysvětli mi, proč je pro střely lepší použít `ArrayList` místo obyčejného pole `[]`. Jak funguje metoda `.add()` pro přidání střely a jak bezpečně odstranit střelu pomocí `.remove()`, když vyletí z obrazovky, aby mi hra nepadala?

Prompt: Jak funguje `addKeyListener` v Javě? Vysvětli mi rozdíl mezi metodami `keyPressed`, `keyReleased` a `keyTyped`. Jak zajistím, aby postava reagovala na stisk šipky okamžitě a bez prodlevy, kterou běžně dělá operační systém při psaní textu?